

Versioned Identity and Outcome Accountability for Agentic Finance

A design proposal for personal-agent payment authority

Working paper · revised draft

September 17, 2026

Abstract

When a person authorizes an agent to spend, what should happen to that authority after the agent’s software changes? A persistent identity can preserve history without settling this authorization question. This paper proposes an application-level design that separates agent lineage, declared software configuration, runtime credentials, and financial mandates. It connects those records to payment, fulfillment, and recovery evidence so that a user can distinguish an authorized transaction from a satisfactory result.

The proposal combines established authorization, workload-identity, provenance, and payment-reconciliation ideas. It does not claim to invent these primitives or establish that competing platforms lack them. A local Scrip prototype exercises parts of the design for an exact, human-approved hotel purchase using simulated providers. It supports version-bound registry checks and distinct payment and outcome records, but does not establish that the declared software actually ran, authenticate real merchant evidence, or demonstrate production payment safety. The principal research question is whether configuration-sensitive authority can reduce harmful inherited permissions without imposing excessive approval friction. That benefit, and consumer demand for it, remain unmeasured.

1. Continuity of identity, continuity of authority

Consider an illustrative delegation: a person allows a travel assistant to book a refundable room for specified dates, with a total ceiling of \$700. Before purchase, the operator changes the assistant’s model and purchasing tools. The assistant retains its name and transaction history. Should its existing permission still apply?

There is no need to decide whether this is philosophically the “same agent.” The narrower engineering question is whether the current execution context satisfies the conditions under which spending was authorized. Identity continuity and authority continuity are separate decisions. A stable identifier is not itself a vulnerability; granting authority solely because that identifier persists would be inadequate in deployments where software changes matter.

This question is already present in industry discussion. Natural’s founder explicitly raises the problem of a stable Agent ID representing changing code, particularly when someone else operates the agent. His essay also distinguishes executing an agent’s financial instruction from responsibility for an unsatisfactory agent outcome. These are useful motivations, not evidence of an undisclosed implementation gap or a contractual interpretation of Natural’s services. [1]

The proposed design makes three links explicit:

1. **Authority to configuration:** which declared versions may use a particular mandate?
2. **Configuration to execution:** what evidence supports the claim that this version acted?
3. **Execution to outcome:** which payment and fulfillment facts support the result reported to the person?

The contribution is a concrete composition of these links and an evaluation agenda. No claim of first invention, complete protocol standardization, or demonstrated competitive superiority is made.

2. Existing work and the scope of the proposal

This is a targeted review of relevant primary documentation, not a systematic literature review or exhaustive market survey. Sources were accessed on September 17, 2026. Vendor statements describe public positioning; specifications describe intended protocol behavior; neither alone proves adoption or operational performance. Drafts and preprints are identified as such.

2.1 Delegation and transaction intent already have substantial coverage

OAuth Rich Authorization Requests already expresses fine-grained authority in structured form, including payment amount and recipient. Granular policy is therefore not an invention specific to AI agents. [2]

The inspected AP2 v0.2 specification defines Checkout and Payment Mandates, linked receipts, verification responsibilities, and direct and autonomous flows. Its constraints are evaluated by designated participants; it also describes evidence used in disputes. The earlier launch essay used Intent and Cart Mandate terminology, so an implementation should follow a pinned specification rather than infer today’s schema from that announcement. This paper’s agent-configuration version is distinct from AP2’s mandate-schema version. [3, 4]

Mastercard-maintained Verifiable Intent, marked draft v0.1 in the inspected repository, specifies a delegation chain, selective disclosure, purchase constraints, and checkout-payment binding. It distinguishes machine-enforceable constraints from descriptive product information and leaves dispute resolution outside its scope. Visa’s Trusted Agent Protocol specifies signed agent-recognition and related commerce messages for participating merchants. Neither a valid message signature nor a signed description should be interpreted as a measurement of an entire executing agent. [5, 6]

2.2 Persistent identities and consumers are not unserved categories

Natural publicly describes persistent agent history, revocable keys, limits, and permissions. Its public materials cover a broad financial stack and discuss disputes; it would be inaccurate to characterize Natural as only a wallet or as lacking guardrails. This review did not verify its internal version-binding implementation, every product’s availability, or an integration with Scrip. [1, 7, 8]

Stripe’s April 2026 announcement describes a Link wallet for agents, consumer approval of spending requests, and one-time cards or Shared Payment Tokens without exposing underlying payment credentials. A “consumer wallet for agents” is thus not sufficient differentiation by itself. [9]

The potential product distinction examined here is narrower: making the relationship between software changes, outstanding authority, and subsequent outcomes understandable to the principal.

Whether that merits a standalone product, a provider feature, or an open-source component is an open commercial question.

2.3 Identity, provenance, and outcome verification have prior art

SPIFFE/SPIRE already separates workload identity from credentials and uses workload properties during attestation. SLSA provenance describes how an artifact was produced. The IETF RATS architecture distinguishes evidence, appraisal, and attestation results, with explicit freshness and trust assumptions. These are foundations to reuse, not missing infrastructure that this paper replaces. [10–12]

ERC-8004, marked draft at review time, proposes identity, reputation, and validation registries while leaving payments orthogonal. Agentic Settlement Protocol is particularly close adjacent work: its September 2026 preprint proposes holds, fulfillment verification, and partial-refund mechanisms for delayed-fulfillment stablecoin commerce, including travel. It identifies itself as a design paper without deployed measurements. Outcome-aware commerce and recovery are therefore not novel merely because they are connected to agents. [13, 14]

This proposal concentrates on principal-approved configuration changes across a mission lifecycle. That emphasis does not establish priority over all prior work, and no interoperability with these systems has been demonstrated.

3. System model and trust assumptions

The **principal** grants authority. An **operator** deploys the agent. A **control service** authenticates requests, checks mandates, and maintains budget and operation state. **Payment providers** report financial events. **Evidence providers** report fulfillment or recovery events. Some roles may belong to one organization, but their claims should remain distinguishable.

The agent may generate mistaken requests, encounter malicious content, repeat operations, or continue after an operator changes its configuration. Providers may time out, return events out of order, or supply inconsistent evidence. These are the failures the design aims to expose or constrain.

The intended controls require all covered spending to pass through the service or equivalent provider-enforced restrictions. An agent with an unrestricted credential outside that boundary can bypass them. The design assumes a trustworthy authorization service, protected state, authenticated principal actions, and appropriate verification of provider messages. A compromised administrator, colluding evidence source, or stolen credential can invalidate those assumptions. Storing a hash beside data does not protect both from an attacker who can rewrite the database.

“Verified outcome” means an assessment supported by specified evidence under a stated trust policy. It does not mean objective truth, customer satisfaction, legal liability, or proof of the agent’s private reasoning.

Manifests and receipts can reveal sensitive instructions, destinations, purchases, and behavior. The proposed service should minimize disclosure, restrict access, and define retention periods. A digest is not encryption: predictable underlying content may still be guessed. Recording private chain-of-thought is neither necessary nor an assurance mechanism in this design.

4. Proposed authority model

4.1 Keep four objects separate

Agent lineage records continuity: an identifier, accountable owner and operator, status, and historical links. Membership in a lineage grants no spending authority on its own.

Agent version records a declared configuration: model reference, whether that reference is mutable, instruction and policy digests, code artifact reference, tool versions, and permissions. A version references its lineage and optionally its predecessor. Changing declared configuration produces a new version record; that rule does not determine whether behavior changed.

Runtime credential authenticates a caller under an issuance policy and binds it to a version record. Credential rotation is a separate event from software change. Possession of a bearer secret establishes access to that secret, not exclusive possession by a particular binary or model.

Financial mandate records the principal, authorized version set, permitted funding source and operations, amount and time constraints, approved purchase or goal conditions, recovery permissions, and revocation state. A production format also needs issuer, audience, schema version, and explicit verification rules. The mandate may be an authenticated server-side record; portable verification additionally requires an appropriate signature or credential scheme.

One useful authorization predicate for a proposed request is:

$$\text{allow}(r) = A(r) \wedge V(r) \wedge M(r) \wedge P(r) \wedge B(r).$$

Here, A means the caller is currently authenticated; V means its version binding meets the required assurance; M means its mandate is active and applicable; P means the structured request satisfies policy; and B means sufficient authority can be reserved atomically. This is a design condition, not a proof that the prototype enforces every term across all entry points.

The service should reject unrecognized mandatory constraints. Natural-language interpretation can propose structured terms, but a model's assertion that a request is appropriate must not substitute for their enforcement. The human approval surface must show material terms, including fees, dates, and recovery limits.

4.2 State the strength of the version binding

Three assurance levels should be distinguished:

- **Declared:** an operator supplies a manifest. Its digest identifies the record, not the running program.
- **Release-bound:** a trusted release or deployment process signs provenance and binds credential issuance to an artifact. Assurance depends on that process and its coverage.
- **Runtime-attested:** an accepted verifier appraises fresh evidence about a measured environment and binds the result to the requesting workload or key.

These are proposed application labels, not interchangeable industry certifications. Build provenance is not runtime attestation. Runtime attestation covers only measured components and the verifier's policy; it cannot automatically attest a remote model provider, changing web content, or merchant fulfillment. Freshness narrows but cannot eliminate the interval in which a system may change after measurement. [11, 12]

A manifest also needs an encoding profile. Implementations must agree on normalization, ordering, absent fields, numeric representation, and algorithm identifiers before exchanging digests. RFC 8785 offers a JSON canonicalization scheme; this paper does not claim that Scrip’s custom serializers conform to it. [15]

4.3 Treat change approval as policy, not semantic proof

The conservative default is an explicit version allow-list: a newly registered version does not inherit authority. Model, instruction, financial-policy, tool, or permission changes require review under the principal’s policy. A dependency patch or logging change is not automatically harmless; apparently incidental code may affect credentials, data exposure, or execution.

Display metadata stored outside the authority-bearing manifest can change without changing the grant. Operator transfer should trigger a separate ownership and authority review. Key rotation may retain a version binding only through an authorized rotation procedure.

An unchanged digest cannot detect undeclared changes, mutable model aliases, new retrieval content, or evolving memory. Short-lived grants, explicit dynamic-input boundaries, and stronger deployment controls may reduce that exposure. They do not establish semantic equivalence. The service should surface this uncertainty instead of displaying “same software” as an unconditional guarantee.

5. Execution, reconciliation, and recovery

The intended sequence is:

Approve terms → authenticate and check authority → reserve budget and record operation → dispatch through a provider → reconcile financial facts → assess fulfillment → pursue authorized recovery.

5.1 Reserve authority before creating external effects

Within one local transaction, the service should record the operation identity and reserve its maximum exposure. This prevents concurrent requests from allocating the same local budget. A reservation is an accounting control, not necessarily a bank hold, escrow deposit, or settled balance.

Before dispatch, the service should recheck current authority. A production design must define how revocation and dispatch are ordered: a check immediately before a network call alone does not establish atomic ordering with a concurrently changing registry. Once a provider accepts an instruction, subsequent revocation may block new instructions without retracting the accepted one.

The request identity should bind the mandate revision, action, and normalized payload. A repeated key with a different payload must be rejected. An ambiguous provider response should leave the operation unresolved until reconciliation or a provider-supported retry establishes its status.

Idempotency is provider-specific, not a universal exactly-once guarantee. Stripe, for example, documents parameter comparison and allows key removal after at least 24 hours; reuse after pruning can create a new request. Local deduplication cannot prevent a provider’s independent duplicate effects or an unmediated purchase. Each adapter needs an explicit retention, retry, reconciliation, and multi-capture contract. [16]

5.2 Payment success and fulfillment success are independent

Capture must be recorded when supported by financial evidence, even if fulfillment fails. Delaying internal recognition of spent money until the desired outcome occurs would misstate exposure. An ordinary card purchase also cannot be assumed to delay settlement until an application approves fulfillment; that would require separately supported commercial and payment arrangements.

For the hotel example, a matching reservation confirmation can support “booking confirmed.” It does not prove that the person completed a satisfactory stay. A broader service-outcome contract would need additional stages, evidence, and deadlines. Merchant identifiers, transaction references, currency, amounts, and event provenance must be checked independently of the agent’s narrative.

A receipt should therefore report at least the approved ceiling, outstanding reservation, recorded capture, reversal, refund requested, refund confirmed, unrecovered amount, and fulfillment assessment. Contradictory evidence should remain visible. New information produces a new assessment or receipt revision; earlier observations should not be silently rewritten.

5.3 Recovery is a workflow, not an undo button

The mandate can permit cancellation or a refund request when a defined failure occurs. It cannot compel a merchant to accept a request or a payment network to reverse a transaction. Refunds may remain pending or fail; Stripe’s documentation explicitly describes those states. A request acknowledgment must not be represented as recovered funds. [17]

Even confirmed returned funds do not automatically renew spending authority. A policy must specify whether recovery restores an allowance, reduces net exposure only, or requires new approval. Otherwise, repeated purchase-and-refund cycles could evade a gross-spend limit. Replacement purchases must account for overlap with unresolved charges.

Dispute preparation can collect evidence, but submission should follow explicit human authorization and provider rules. This design supplies neither legal advice nor a guarantee of reimbursement, insurance, custody compliance, or consumer-protection coverage.

6. What the Scrip prototype actually establishes

The inspected local Scrip checkout contains a TypeScript mission service and SQLite-backed stores. It implements an exact, human-approved hotel purchase, not the full multi-category personal-wallet product described in its specification. The observations below come from source inspection and local tests, not live payment execution.

Agent identity types define lineage, version manifests, credentials, and financial mandates. The registry stores those records and provides secret authentication, version allow-list checks, revocation epochs, successor-change handling, and credential rotation. The mission service checks agent-bound missions before reservation and again before purchase execution, records operation state before provider calls, and reconciles outcomes. The identity binding is optional for legacy unbound missions.

The outcome assessor checks structured hotel and payment observations against the approved contract. It does not treat the execution provider’s narrative as fulfillment evidence. However, accepted booking records carry caller-supplied merchant or email source labels; this alone does not

authenticate the source. Provider authenticity and independence remain adapter and deployment obligations.

Several boundaries prevent stronger claims:

- **No measured-runtime guarantee.** The registry defaults to self-declared assurance. Its registration API also accepts an attestation-level argument without performing an attestation procedure. Stronger labels are not evidence of stronger assurance.
- **No hardened perimeter demonstrated.** The mission API accepts an authenticated-agent object from its caller. A TypeScript type is not a runtime security boundary; trusted wiring must ensure that object came from credential verification. Registration and approval methods likewise require an authenticated surrounding service.
- **Stored fields are not all enforced predicates.** For example, the registry's authorization method checks version, principal, contract digest, time, and status, but does not itself validate funding-source and operation-scope fields against a dispatch request.
- **No global revocation theorem.** There is a recheck before purchase execution, but payment-capability issuance occurs earlier. The registry and mission stores do not establish one atomic transaction with the external provider. Credential freshness at dispatch and race behavior need further scrutiny.
- **No public reproducible release or live-provider result.** Mission and identity files were untracked in the inspected working tree. The Git HEAD alone does not identify this prototype. No real booking, refund, provider conformance run, or production deployment was verified in this review.

These are limits of the current evidence, not reasons to discard the design. The useful demonstrated slice is local orchestration: explicit approval, reservation before provider I/O, version-associated authority checks, and separation of payment observations from fulfillment assessment.

Local verification record

On September 17, 2026, the test command shown in the appendix exited 0: **247 passed, 8 skipped**, across 21 passing test files and one skipped file. The identity file contained 21 passing tests; the purchase-mission file contained 50. The eight skipped tests were the PostgreSQL store suite. The TypeScript check also exited 0.

Those totals describe the entire checked-out test suite, not 247 independent validations of this paper. Passing fixtures do not prove security, general payment correctness, real evidence authenticity, or fulfillment reliability. No financial operations were initiated by this review.

7. Evaluation required before stronger claims

The next study should compare three configurations using the same tasks and providers: transaction-scoped controls alone; those controls plus declared version binding; and version binding backed by a measured deployment. This would isolate the value of configuration awareness from benefits already supplied by a strong mandate system.

Authority tests should include undeclared configuration changes, mutable aliases, stolen and expired credentials, forged caller objects, operator changes, version rollback, missing scopes, and revocation during capability issuance and dispatch. Report both blocked unauthorized operations and legitimate operations incorrectly blocked.

Failure-injection tests should interrupt each persistence and network boundary, reorder or duplicate events, reuse keys after provider retention windows, and simulate partial captures, provider inconsistency, and refund failure. Report unresolved exposure and reconciliation time rather than assuming every timeout can be safely retried.

Evidence tests should include fabricated merchant labels, compromised adapters, conflicting confirmations, cancelled bookings, and stale refund events. Report false success, false failure, and unresolved outcomes against independently established ground truth.

Usability tests should measure whether people understand the approved authority, the reason for a change prompt, and the difference between “refund requested” and “money returned.” Approval count, completion time, abandonment, and misunderstanding should be reported alongside safety outcomes. Frequent approval prompts could make the system less useful or encourage indiscriminate consent.

The central hypothesis would be weakened if version binding provides little additional protection over scoped transaction mandates, or if its approval burden outweighs that protection. Reputation-based increases in authority require a separate study: past outcomes must not silently become permission, and lineage-level success may not predict a new version’s behavior. No reputation model or consumer study is evaluated here.

8. Integration and product implications

The design could be implemented within an existing payment platform or in a principal-facing application using one. It does not inherently require a new payment rail, token, blockchain, or legal identity standard. A personal agent can present one continuing identity while exposing changes to the authority-bearing configuration underneath.

An AP2 integration would need an agreed constraint schema and verification algorithm for version binding, not merely an extra JSON field. A workload-identity integration would need explicit credential-issuance and attestation policy. A payment adapter would need provider-specific rules for limits, authorization expiry, settlement, refunds, and disputes. Fields that cannot be faithfully enforced on a rail should lead to a restricted mode or refusal, not a claim of identical protection everywhere.

Natural’s discussion of changing agent identity makes this a relevant design question to explore with its team. It does not establish a feature Natural has overlooked. The constructive next step is a narrowly scoped integration experiment and technical feedback, not a claim that Scrip replaces Natural’s stack. Access, provider agreement, protocol compatibility, and commercial feasibility remain unverified.

The consumer hypothesis is that people value an understandable record of what they authorized, which agent configuration acted, and what happened afterward. Public consumer-wallet offerings already exist. The paper presents neither market-size estimates nor evidence that consumers will pay separately for this record and its controls.

9. Conclusion

An agent can retain its identity while its permissions are reconsidered. A payment can be authorized while its outcome remains unresolved. Keeping these distinctions explicit produces a useful design discipline for personal-agent finance.

The proposed model binds mandates to declared versions, makes assurance strength visible, and follows operations through financial and fulfillment evidence. Existing standards provide much of the foundation. The local Scrip prototype offers a limited implementation of the orchestration model, not proof of runtime identity or production financial safety. The next contribution should be evidence: an authenticated integration, adversarial failure tests, and measured benefits relative to transaction-scoped controls.

References

Primary sources below were accessed September 17, 2026. Repository branches and product documentation can change; versions are stated where visible. Source descriptions are not endorsements or independently audited product claims.

- [1] Kahlil Lalji / Natural. [Agentic payments, revisited](#). August 12, 2026. Founder essay; identity, liability, and platform sections.
- [2] T. Lodderstedt, J. Richer, and B. Campbell. [RFC 9396: OAuth 2.0 Rich Authorization Requests](#). 2023. Standards-track specification.
- [3] Google Agentic Commerce. [AP2 specification](#). Inspected document labels itself v0.2; moving repository branch.
- [4] Google Cloud. [Announcing Agent Payments Protocol](#). Launch announcement; consulted for historical terminology, not as the current schema.
- [5] Mastercard / agent-intent contributors. [Verifiable Intent](#). Draft v0.1, as labeled in the inspected repository.
- [6] Visa. [Trusted Agent Protocol specifications](#). Merchant-facing protocol documentation.
- [7] Natural. [Identity](#). Public product description.
- [8] Natural. [Disputes](#). Public product description.
- [9] Dan Hill / Stripe. [Giving agents the ability to pay](#). April 29, 2026. Link wallet product announcement.
- [10] SPIFFE. [SPIRE concepts](#). Workload registration and attestation documentation.
- [11] SLSA. [Provenance, specification v1.2](#). Software supply-chain provenance.
- [12] H. Birkholz et al. [RFC 9334: Remote ATtestation procedureS Architecture](#). 2023. Informational architecture; especially trust and freshness.
- [13] M. De Rossi et al. [ERC-8004: Trustless Agents](#). Draft, created August 13, 2025; status checked at review time.
- [14] B. Mohammadkhani, A. Khekade, and R. Kakkad. [Agentic Settlement Protocol: An Application Profile for Refundable, Delayed-Fulfilment Agent Commerce on Stablecoin Rails](#). arXiv:2609.02208v1, September 2, 2026. Preprint/design proposal; no peer-review claim made here.
- [15] A. Rundgren, B. Jordan, and S. Erdtman. [RFC 8785: JSON Canonicalization Scheme](#). 2020. Informational specification.
- [16] Stripe. [Idempotent requests](#). API documentation; key retention and parameter matching.

[17] Stripe. [Refunds](#). Operational documentation; pending and failed refund states.

Appendix: prototype evidence boundary

The local verification used the Scrip working checkout (project directory: spending).

Commands executed, both exit status 0:

```
npm test -- --run
npx tsc --noEmit
```

Git HEAD was:

```
8ce4e6f09e4bd4ebc7c07242dd33c69ce84edf1c
```

The checkout contained pre-existing modified and untracked files. HEAD is not a release identifier for the tested mission implementation. Source paths below are relative to that checkout.

- Agent identity and manifest classification: **src/missions/agent-identity.ts**.
- Authentication and authority: **SqliteAgentRegistry** in **src/missions/agent-registry.ts**, especially **registerVersion()**, **authenticate()**, and **authorize()**.
- Orchestration: **PurchaseMissionService** in **src/missions/purchase-mission-service.ts**, especially **execute()** and **reconcile()**.
- Assessment: **assessOutcome()** in **src/missions/outcome-assessor.ts**.
- Identity fixtures: **tests/agent-identity.test.ts**.
- Purchase fixtures: **tests/purchase-mission-service.test.ts**.

SHA-256 digests below identify four inspected files, not a complete reproducibility package. Each digest is split across two lines for print; concatenate them without whitespace.

```
agent-identity.ts
1e78443d8622fa050e4154fab609e8307
b045f89200573d621f1792929f06c21
```

```
agent-registry.ts
62feeafff82de6c9cfc9e64099744f966
fef8e75d2ac58be34d90f1d75ca338d
```

```
purchase-mission-service.ts
60833e10e717dda9b0fe19eb38f803701
3523e8e84554066edb5d4981f16f8be
```

```
outcome-assessor.ts
28301496041a5bd58a7a6ef7097f689ef
e7979afe72a6cfe384ce631380d2b2f
```

External replication requires publishing a coherent source snapshot, dependency lockfile, runtime version, fixtures, and commands. This revision supplies local test evidence and explicit limitations; it does not claim that requirement has been met.